CSC_51053_EP, Exercise Session 1: Data Modeling

Ioana Manolescu, Madhulika Mohanty

September 26, 2025

The transition from natural language to an E-R diagram often has a fuzzy aspect, given that we start with natural language, and that some aspects of the real-world application may or may not be thoroughly specified. Further, the E-R model inherently has some freedom degrees.
For both these reasons, in general, *there may be several correct E-R diagrams for a given problem.*

1 Entrance exams

Admission in French engineering schools is done by exams. Each candidate has a name and an ID. Each school admits candidates through one or several tracks (or majors), e.g., "Math-Physics", "Physics-Chemistry", etc. For each track, the school publishes a number of places. To be admitted in a track, candidates must first pass a fixed set of written exams. Each exam has a name, e.g., "Maths 1", and an exam topic. For each written exam, each candidate gets a grade. Also, for each written exam of one school, the school has a coefficient (weight) to give to the exam. For each track, a fixed number of candidate will be admitted to pursue to oral exams. Concretely, those candidates are those whose weighted averages (sum of grades $\times$ respective coefficients) are the highest.

Each school also determines a set of oral exams to be taken by any candidate remaining after the written exams. Each oral exam has a weight, and each candidate gets a grade. A candidate gets an offer from a school in a track, if the candidate has applied to that track of the school, and the candidate's overall grade (the sum of the candidate's oral and written exam grades, each multiplied by the respective weight) ranks the candidate at the top (within the number of places).

1. Propose a minimal Entity-Relationship model describing the admission process. Which aspects of the specification are not described in the model? Briefly explain why.

   A minimalist set of entities could be: Candidate(cid, cname), Track(tid, name, admissionCapacity, oralCapacity), Exam(eid, name, topic, coefficient), WrittenExam isA Exam, OralExam isA Exam. Relationships: examInTrack between Exam and Track, each Exam is in exactly one relationship, each Track has at least one Exam; candidateTakesOralExam between Candidate and OralExam, with a grade attribute ; candidateTakesWrittenExam between Candidate and WrittenExam, with a grade attribute grade; each Candidate takes at least one WrittenExam (otherwise, they are not really candidates).
   Notes:

   (a) The above does not even model the schools, but observe that they are not strictly required (given that admission is per-track);

   (b) We do not model directly model that a candidate applies to a track, but only indirectly (via the exams); as a minimalistic solution, this is fine;

   (c) We do not model the real-world constraint that a candidate must take all the exams in a track. With the entities above, our E-R model does not capture this. Note that in practice a candidate may fail to show up for an exam and still get an offer, because there is a 0 grade for each non taken exam. Our minimalistic model does not capture that.

   (d) We do not model how the decisions are taken, but there is enough information in our model to take those decisions, and to store their outcome; this illustrates that *E-R models are not, in general, meant to store computation logic.*

   (e) We do not model that a candidate takes an oral exam only if the candidate took at least some written exams.

2. If we want to record the date when each candidate takes an exam, where can we put it? As an attribute to candidateTakesOralExam, respectively, candidateTakesWrittenExam.

3. Suppose we want to add to the model the fact that to apply to a track of a school, a candidate must pay a fee that is the same for all the tracks of the school. Modify the above model to reflect that. Add a School entity with a fee attribute, attach the Track to the School (each track is attached to exactly a school, each school has at least one track).

4. How can we change the model so that we make sure when a candidate registers to a track, the candidate has a grade for each written exam in the track? Remove the candidateTakesWrittenExam relation and instead have a relation candidateAppliesToTrack between Candidate and Track, with the attributes gradeExam1, gradeExam2, gradeExam3, etc. for each writtenExam. Note that:

   - Creating the relationship requires knowing all the grades, which is not very practical (we would like to create the relationship as soon as we have the first grade, but at that time, the other grades are not available!) In practice, of course, this is solved with a default grade, e.g., 0, for all exams. Then, when the grade to exam2 becomes available, we would model this as an update to the respective instance of the candidateTakesWrittenExam relationship.
   - This solution increases the size of the E-R model by a factor that is "the instance size" in the first model: the number of written exams of a track.

2 E-commerce database

We consider an online commerce site selling *products* (supplied by a set of *vendors*) to *clients*. Each product has an *ID*, a *name* and a *description*. Each vendor has an *ID* and an *address*. Each client has an *ID*, a *name* and an *address*. Each vendor supplies at least one product. For each product that it supplies, a vendor charges a certain *price*, and some *shipping fees*, which are determined by the address of the vendor, the address of the client, and the product itself. The *delivery time* of a product to a client is also dictated by the vendor and client addresses. The online commerce site charges clients a *margin*, which is a percentage of the price of every product they buy. This percentage is the same for all the products supplied by a given vendor but it can be different for diffent vendors.

1. **(E-R model)** Propose a set of entities and relationships that model the data described above and its constraints. If you detect constraints which cannot be expressed using the notions seen in the course, state the constraint and briefly explain why.

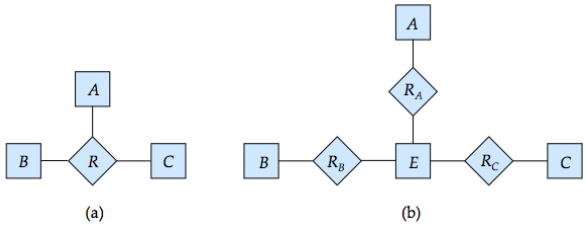
   Entities:
   Product(pID, pName, pDescription); Vendor(vID, vAddress, charge) where charge is the percentage charged by the online site; Client(cID, cName, cAddress).
   Relationships: Supplies between Vendor and Product, with the attribute price, and participation constraints on both sides;
   BuysFrom between Client, Product and Vendor, with the attributes shipping fees and delivery time.
   Missing constraint: a client can only buy from a Vendor a product that the vendor actually supplies.

2. **(Relational model)** Transform this Entity-Relationship model into tables (relations) and constraints as seen in the course. Use single-table constraints whenever possible and multiple-table constraints only if necessary.

   Tables with underligned primary keys: Product(pID, pName, pDescription); Vendor(vID, vAddress, charge); Client(cID, cName, cAddress); Supplies(vID, pID, price), vID foreign key in Vendor.vID, pID foreign key in Product.pID; BuysFrom(cID, pID, vID, fees, deliveryTime) with three foreign keys on Client.cID, product.pID, Vendor.vID.
   The extra constraint "a client only buys from a Vendor a product that the vendor actually supplies" should be enforced by looking into Supplies and BuysFrom. In particular, when inserting a BuysFrom tuple, we should check that (vID, pID) was already in Supplies: this can be achieved by making (vID, pID) foreign key references Supplies.
   The same can be achieved with a constraint: `check (vID, pID) in (select vID, pID from Supplies)`; however, this class of constraints is not supported by Postgres. This is why a FK is preferrable as a more generally supported mechanism.

3 Exercise 2 extended from [1]: modeling ternary relationships

Consider the two alternatives below for modeling a ternary relationship R between entities of the sets A, B and C using E/R schema: The first diagram (a) is a straightforward translation of a ternary relation. The second diagram (b) is a more complex translation of R.



1. Construct a simple instance for the entity sets E, A, B, C and relationship sets R_A, R_B, R_C, which *cannot* be modeled with the entity sets A, B, C and the relationship R.

   The first diagram requires to have a relationship among all entities. The second diagram allows to record partial relations between less than three entities. Concretely, consider a data instance consisting of: $R_A(a_1, e_1)$, $R_B(b_1, e_1)$ and no instance of R_C. An equivalent instance cannot be created for the first E-R model, since we cannot define an instance of the form $R(a_1, b_1, N)$.

2. Propose a **diagram (b) variant (modification)** to introduce constraints that guarantee that any instance over the entities E, A, B, C and relationships R_A, R_B, R_C which satisfies the constraints, can be modeled with the entity sets A, B, C and the relationship R.

   1. Add *a key and a participation (thus, an exactly one) constraint* on R_A, R_B and R_C to E: Any E instance must participate to exactly one of these. Thus, any instance of E determines (corresponds to) an (A, B, C) triple.
   2. Let us also consider an alternative: *add a participation constraint on R_A, R_B and R_C to E (but not a key constraint).* This also suffices, e.g., consider the following instance of the modified schema in (b):

   $$R_A(a_1, e),\ R_A(a_2, e),\ R_B(b_1, e),\ R_B(b_2, e),\ R_C(c, e) \qquad (\diamond)$$

   where e participates to two instances of R_A and to two instances of R_B. We can translate this instance into the model (a), as follows:

   $$R(a_1, b_1, c),\ R(a_1, b_2, c),\ R(a_2, b_1, c),\ R(a_2, b_2, c) \qquad (\circ)$$

3. Now consider that in (a), entities A, B, C all have a total participation constraint to the relationship R. Modify the result of the previous point (variant on (b)) in order for the three-entities model to capture any instance of the four-entities one. Provide two variants:

   (a) Using participation constraints;

   In the modified (b) diagram, add to A a participation constraint to R_A and similarly for B and R_B, C and R_C.

   (b) Using weak entities.

   In the (b) diagram variant, make A, B and C weak entities, with R_A, R_B and R_C their identifying relationships. This ensures each A, B, C entity instance is associated with exactly one E instance (constraint on weak entities), and this, together with the change already made in (b) above, ensures any E instance determines an A, B, C triple *and* any A, B or C is connected to one E. This amounts to total participation of A, B and C to R.
   The correctness of this solution can be verified by writing and checking the respective FOL formulas of the form $\forall a \in A$ etc.

4. As a variant on this exercise, consider *four* entities A, B, C and D, and a relationship S of arity four between them. Assume we want to express the fact that *as soon as four entity instances a, b, c, d are connected through S, the entities a, b, c are also connected through R.*

(a) Can this be done by adding entities and/or relationships to the ER model (a) at the beginning of this exercise?

We can easily add the relationship S of arity four to the diagram. However, the Entity-Relationship model does not include a construct to state inclusion constraints between relationship sets, and this is exactly what we would need here. Thus, if we *create R and S explicitly as relationships*, there is no way to achieve the desired goal.

(b) Can this be done by adding entities and/or relationships to the ER model (b), modified as above in item 3a?

The solution presented below is based on a way to *encode* (simulate) S instead of actually creating it, as follows.
Create a new binary relationship between E, the central entity, and D; call this relationship R_D. Remember that in the modified (b) model we considered, an instance of E corresponds to exactly one (a, b, c) triple. If we want to model that four instances $a \in A$, $b \in B$, $c \in C$, $d \in D$ are connected through S, we create a new e such that each e is uniquely associated with an (a, b, c) combination, connect it to a, b and c through R_A, R_B and R_C respectively, and connect d and e through R_D. It is easy to see that this representation satisfies the given requirement.

References

[1] H. Korth, S. Sudarshan, and A. Silberschatz. *Database System Concepts (6th ed.)*. McGraw-Hill Education, 2010.